

Machine Learning Model Testing and Risk Prediction

Author: Dominique Ball

Corresponding Author Email: Domhasaball@gmail.com

Article Type: Viewpoint / Tutorial

1. Introduction

The purpose of this test was to evaluate the functionality of a trained machine learning model that predicts a patient's risk level based on a set of physiological and demographic measurements. The model is stored as a serialized `.joblib` file and is loaded into the Python environment using the `joblib` library.

The prediction system uses seven patient features: age, heart rate, systolic blood pressure, diastolic blood pressure, respiratory rate, oxygen saturation (SpO₂), and temperature. The model produces a probability representing the predicted risk, which is then converted into one of three risk categories: Lower Risk, Intermediate Risk, or Higher Risk.

2. Model Loading

The model is stored in the project's `models` directory as `model.joblib`. The program uses Python's `pathlib` library to construct the path to the model relative to the project directory.

Before loading the model, the program checks whether the model file exists. If the file cannot be found, a `FileNotFoundError` is raised with the expected file location. This provides a basic safeguard against attempting to run predictions without a trained model being available.

The model is loaded using:

```
joblib.load(MODEL_PATH)
```

This allows the previously trained machine learning model to be reused for making predictions on new patient data.

3. Input Features

The prediction function accepts seven input variables:

Feature	Description
Age	Patient's age
Heart Rate	Patient's heart rate
Systolic BP	Systolic blood pressure
Diastolic BP	Diastolic blood pressure
Respiratory Rate	Patient's respiratory rate
SpO ₂	Blood oxygen saturation
Temperature	Patient's body temperature

These variables are placed into a Pandas DataFrame. The DataFrame columns are then explicitly reordered according to the predefined **FEATURES** list.

This ordering is important because the machine learning model expects the input variables to correspond to the same feature structure used during model training.

4. Prediction Process

Once the patient data has been prepared, the model's `predict_proba()` method is used to generate probability estimates.

The code retrieves the probability associated with class `1`:

```
probability = model.predict_proba(patient)[0][1]
```

The resulting value is converted to a Python floating-point number and returned by the `predict_risk()` function.

For example, if the model produces a probability of `0.82`, this means that the model assigns an estimated probability of 82% to the positive/risk class, assuming that class `1` was defined as the positive risk outcome during model training.

5. Risk Classification

The predicted probability is converted into a risk category using three predefined thresholds:

- **Probability ≥ 0.70 :** Higher Risk
- **Probability ≥ 0.40 and < 0.70 :** Intermediate Risk
- **Probability < 0.40 :** Lower Risk

The classification logic is implemented as follows:

```
if probability >= 0.70:
```

```
    return "HIGHER RISK"
```

```
if probability >= 0.40:
```

```
    return "INTERMEDIATE RISK"
```

```
return "LOWER RISK"
```

This approach provides an easy-to-understand interpretation of the model's numerical output.

6. Example Test

A test patient can be passed to the `predict_risk()` function using values for all seven required features. The model then returns a probability between 0 and 1.

For example, if the model returns:

Predicted probability = 0.76

the classification function will assign:

Risk classification = HIGHER RISK

Similarly, a probability of 0.55 would result in **INTERMEDIATE RISK**, while a probability of 0.25 would result in **LOWER RISK**.

7. Results and Interpretation

The test demonstrates that the prediction pipeline can successfully:

1. Locate and load the trained machine learning model.
2. Accept the required patient measurements.
3. Organize the measurements into the expected feature format.
4. Generate a probability using the trained model.
5. Convert the probability into an interpretable risk category.

The output of the system should be interpreted as a **model prediction rather than a medical diagnosis**. The reliability of the prediction depends on the quality and representativeness of the training data, the model's performance, and whether the test inputs fall within the range of data used during training.

8. Limitations

There are several limitations that should be considered when evaluating the test.

First, the code assumes that the model was trained using the same seven features and in the same format. A mismatch between training and prediction features could produce incorrect results.

Second, the thresholds of 0.40 and 0.70 are manually defined. These thresholds should ideally be supported by model evaluation and the intended application rather than being selected arbitrarily.

Third, the probability produced by `predict_proba()` should not automatically be interpreted as a clinically accurate probability unless the model has been appropriately calibrated and validated.

Finally, a single test prediction is not sufficient to establish model performance. A more complete evaluation should include a separate test dataset and metrics such as accuracy,

precision, recall, F1-score, ROC-AUC, confusion matrix results, and, where appropriate, probability calibration.

9. Conclusion

The implemented testing pipeline provides a straightforward method for using a trained machine learning model to estimate patient risk. It successfully prepares the patient information in the expected format, generates a probability using the trained model, and translates that probability into three understandable risk categories.

However, further testing and validation are required before the system could be considered reliable for real-world use. In particular, the model should be evaluated on unseen data, its classification thresholds should be justified, and its probability estimates should be assessed for calibration. The system should therefore be considered a machine learning risk-estimation tool rather than a standalone diagnostic system.